

**Unit: III**  
**Physical Devices**  
**Lecture: 3**  
**Basic File System Operations**

A file is a named collection of related information that is recorded on secondary storage, usually as a sequence of bytes, with two views:

- Logical(programmer) view, as the users see it (i.e. how they are used and what properties they have).
- Physical (OS) view, as it actually resides on secondary storage.

In general, a file is a sequence of bits, bytes, lines or records, the meaning of which is defined by the file's creator and user.

A file has a certain defined structure, which depends on its type.

- A text file is a sequence of characters organized into lines (and possibly pages).
- A source file is a sequence of subroutines and functions, each of which is further organized as declarations followed by executable statements.
- An object file is a sequence of bytes organized into blocks understandable by the system's linker.
- An executable file is a series of code sections that the loader can bring into memory and execute.

A file is an abstract data type. To define a file properly, we need to consider the operations that can be performed on files.

**Six basic file operations:**

- **Creating a file:** Two steps are necessary to create a file.
  1. Space in the file system must be found for the file.
  2. An entry for the new file must be made in the directory.
- **Writing a file.** To write a file, we make a system call specifying both the name of the file and the information to be written to the file. The system must keep a write pointer to the location in the file where the next write is to take place. The write pointer must be updated whenever a write occurs.
- **Reading a file.** To read from a file, we use a system call that specifies the name of the file and where (in memory) the next block of the file should

be put. The system needs to keep a read pointer to the location in the file where the next read is to take place.

- Because a process is usually either reading from or writing to a file, the current operation location can be kept as a per-process current-file-position pointer.
- Both the read and write operations use this same pointer, saving space and reducing system complexity.
- **Repositioning within a file.** The directory is searched for the appropriate entry, and the current-file-position pointer is repositioned to a given value. Repositioning within a file need not involve any actual I/O. This file operation is also known as a file seek.
- **Deleting a file.** To delete a file, we search the directory for the named file. Having found the associated directory entry, we release all file space, so that it can be reused by other files, and erase the directory entry.
- **Truncating a file.** The user may want to erase the contents of a file but keep its attributes. Rather than forcing the user to delete the file and then recreate it, this function allows all attributes to remain unchanged (except for file length) but lets the file be reset to length zero and its file space released.

### **Protection in File System:**

There are different methods used by different users to access any file.

The general way of protection is to associate *identity-dependent access* with all the files and directories a list called access-control list (ACL) which specify the names of the users and the types of access associate with each of the user.

The main problem with the access list is their length. If we want to allow everyone to read a file, we must list all the users with the read access. This technique has two undesirable consequences:

Constructing such a list may be tedious and unrewarding task, especially if we do not know in advance the list of the users in the system.

Previously, the entry of the any directory is of the fixed size but now it changes to the variable size which results in the complicated space management.

These problems can be resolved by use of a condensed version of the access list. To condense the length of the access-control list, many systems recognize three classifications of users in connection with each file:

- **Owner:** Owner is the user who has created the file.
- **Group:** A group is a set of members who has similar needs and they are sharing the same file.
- **Universe:** In the system, all other users are under the category called universe.

The most common recent approach is to combine access-control lists with the normal general owner, group, and universe access control scheme. For example: Solaris uses the three categories of access by default but allows access-control lists to be added to specific files and directories when more fine-grained access control is desired.

### **Other Protection Approaches:**

The access to any system is also controlled by the password.

The use of passwords has a few disadvantages:

- The numbers of passwords are very large so it is difficult to remember the large passwords.
- If one password is used for all the files, then once it is discovered, all files are accessible; protection is on all-or-none basis.